

❖ 处理冲突的方法

常用的处理冲突的方法有两种：开放地址法和链地址法。

❖ 开放定址法（Open Addressing）

【定义】所谓开放定址法，就是一旦产生了冲突，即该地址已经存放了其它数据元素，就去寻找另一个空的散列地址。

✎ 若发生了第 i 次冲突，则散列表中地址将增加 d_i ，基本公式是：
空的散列地址是否总能找到？

$$h_i(\text{key}) = (h(\text{key}) + d_i) \bmod \text{TableSize} \quad (1 \leq i < \text{TableSize}) \quad (\text{公式5.10})$$

✎ d_i 决定了不同的解决冲突方案：线性探测、二次探测、双散列。

$$d_i = i$$

$$d_i = \pm i^2$$

$$d_i = i * h_2(\text{key})$$

1. 线性探测法 (Linear Probing)

❖ 即线性探测法以增量序列 $1, 2, \dots, (\text{TableSize} - 1)$ 循环试探下一个存储地址。

[例5.6] 设关键词序列为 {47, 7, 29, 11, 9, 84, 54, 20, 30},

- 散列表表长 $\text{TableSize} = 13$,
- 装填因子 $\alpha = 9/13 \approx 0.69$;
- 散列函数为: $h(\text{key}) = \text{key} \bmod 11$ 。
- 用线性探测法处理冲突, 列出依次插入后的散列表,
- 并估算查找性能。

关键词 (key)	47	7	29	11	9	84	54	20	30
散列地址 $h(\text{key})$	3	7	7	0	9	7	10	9	8

关键词 (key)	47	7	29	11	84	54	20	30						
散列地址 (key)	0	1	2	3	4	5	6	7	8	9	10	11	12	说明
冲突次数	0	0	1	0	0	0	0	3	1	3	1	3	6	

“一次聚集 (Primary clustering)”现象：需要经过9很多次冲突才找到空位置

问题：如何删除关键字“7”？

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	说明
操作														
插入47				47										无冲突
插入7				47				7						无冲突
插入29				47				7	29					$d_1 = 1$
插入11	11			47				7	29					无冲突
插入9	11			47				7	29	9				无冲突
插入84	11			47				7	29	9	84			$d_3 = 3$
插入54	11			47				7	29	9	84	54		$d_1 = 1$
插入20	11			47				7	29	9	84	54	20	$d_3 = 3$
插入30	11	30		47				7	29	9	84	54	20	$d_6 = 6$

表5.1 线性探测法构建散列表的过程

❖ 在散列表中查找数据对象的**平均查找长度 (ASL)**。
 分成**成功查找的ASL (ASL_s)**和**不成功查找的ASL (ASL_u)**。

【分析】**ASL_s**：假设要查找的关键词一定在散列表中存在。只要对查找表中的每个关键词的比较次数加起来，除以关键词的个数，就得到平均每个关键词的查找长度。而每个关键词的比较次数是**其冲突次数加1**。

$$ASL_s = (1+7+1+1+2+1+4+2+4) / 9 = 23/9 \approx 2.56$$

散列表：

H(key)	0	1	2	3	4	5	6	7	8	9	10	11	12
key	11	30		47				7	29	9	84	54	20
冲突次数	0	6		0				0	1	0	3	1	3

【分析】**ASL_u**：假设要查找的关键词一定不在散列表中，并且所有关键词对应的散列地址**均匀分布在地址空间**上。每个地址的探测次数加和/地址空间大小就是**ASL_u**（插入元素的ASL）：

$$ASL_u = (3+2+1+2+1+1+1+9+8+7+6+5+4) / 13 = 50/13 \approx 3.85$$

2. 平方探测法 (Quadratic Probing)

❖ 即平方探测法以增量序列 $1^2, -1^2, 2^2, -2^2, \dots, q^2, -q^2$ 且 $q \leq \lfloor \text{TableSize}/2 \rfloor$ 循环试探下一个存储地址。

有定理显示：如果散列表长度TableSize是某个 $4k+3$ （ k 是正整数）形式的素数时，平方探测法就可以探查整个散列表空间。

[例5.7] 设关键词序列为 {47, 7, 29, 11, 9, 84, 54, 20, 30}

- 散列表表长TableSize = 11（即满足 $4 \times 2 + 3$ 形式的素数）
- 装填因子 $\alpha = 9/11 \approx 0.82$
- 散列函数为： $h(\text{key}) = \text{key} \bmod 11$
- 用平方探测法处理冲突，列出依次插入后的散列表
- 并估算ASLs

关键词 key	47	7	29	11	9	84	54	20	30
散列地址h(key)	3	7	7	0	9	7	10	9	8

关键词 key	47	7	29	11	9	84	54	20	30
散列地址 h(key)	3	8	10	10	8	2	0	9	8
冲突次数	0	0	1	0	0	2	0	3	3

“二次聚集 (Secondary Clustering)”现象：散列到同一地址的那些数据对象将探测相同的备选单元。

$ASL_s = (1+1+2+1+1+3+1+4+4) / 9 = 18 / 9 = 2$

地址 操作	0	1	2	3	4	5	6	7	8	9	10	说明
插入47				47								无冲突
插入7				47				7				无冲突
插入29				47				7	29			$d_1 = 1$
插入11	11			47				7	29			无冲突
插入9	11			47				7	29	9		无冲突
插入84	11			47			84	7	29	9		$d_2 = -1$
插入54	11			47			84	7	29	9	54	无冲突
插入20	11		20	47			84	7	29	9	54	$d_3 = 4$
插入30	11	30	20	47			84	7	29	9	54	$d_3 = 4$

表5.2 平方探测法构建散列表的过程

```
HashTable InitializeTable( int TableSize )
```

```
{
```

```
    HashTable H;
```

```
    int i;
```

```
/* 1*/    if ( TableSize < MinTableSize ){
```

```
/* 2*/        Error( "散列表太小" );
```

```
/* 3*/        return NULL;
```

```
    }
```

```
/* 分配散列表 */
```

```
/* 4*/    H = malloc( sizeof( struct HashTbl ) );
```

```
/* 5*/    if ( H == NULL )
```

```
/* 6*/        FatalError( "空间溢出!!!" );
```

```
/* 7*/    H->TableSize = NextPrime( TableSize );
```

```
/* 分配散列表 Cells */
```

```
/* 8*/    H->TheCells = malloc( sizeof( Cell ) * H->TableSize );
```

```
/* 9*/    if( H->TheCells == NULL )
```

```
/*10*/        FatalError( "空间溢出!!!" );
```

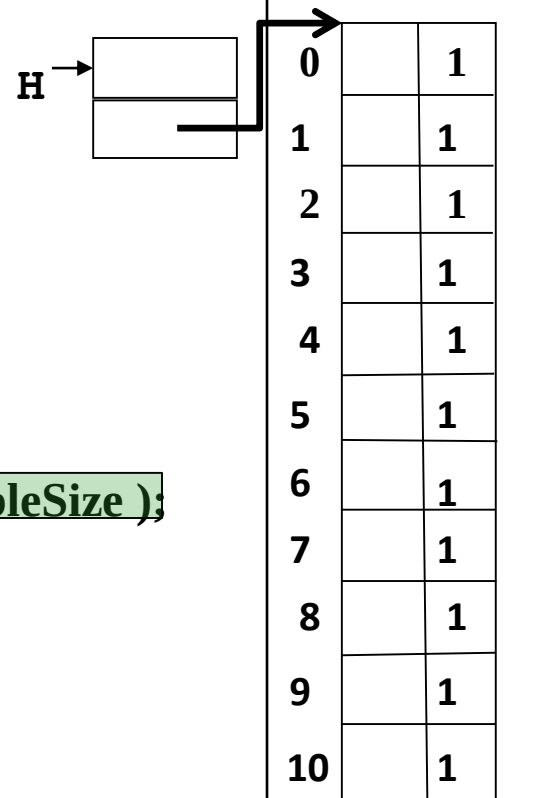
```
/*11*/    for( i = 0; i < H->TableSize; i++ )
```

```
/*12*/        H->TheCells[ i ].Info = Empty;
```

```
/*13*/    return H;
```

```
}
```

```
typedef struct HashTbl *HashTable;
struct HashTbl{
    int TableSize;
    Cell TheCells;
}H;
```



```
Position Find( ElementType Key, HashTable H )
{
    Position CurrentPos, NewPos;
    int CNum;      /* 记录冲突次数 */
/* 1*/    CNum = 0;
/* 2*/    NewPos = CurrentPos = Hash( Key, H->TableSize );
/* 3*/    while( H->TheCells[ NewPos ].Info != Empty &&
               H->TheCells[ NewPos ].Element != Key ) {
        /* 字符串类型的关键词需要 strcmp 函数!! */
/* 4*/        if(++CNum % 2){ /* 判断冲突的奇偶次 */
/* 5*/            NewPos = CurrentPos + (CNum+1)/2*(CNum+1)/2;
/* 6*/            while( NewPos >= H->TableSize )
/* 7*/                NewPos -= H->TableSize;
        } else {
/* 8*/            NewPos = CurrentPos - CNum/2 * CNum/2;
/* 9*/            while( NewPos < 0 )
/* 10*/                NewPos += H->TableSize;
        }
    }
/* 11*/    return NewPos;
}
```

```
void Insert( ElementType Key, HashTable H )
{
    /* 插入操作 */
    Position Pos;
    /* 1*/ Pos = Find( Key, H );
    /* 2*/ if( H->TheCells[ Pos ].Info != Legitimate ) {
        /* 确认在此插入 */
    /* 3*/     H->TheCells[ Pos ].Info = Legitimate;
    /* 4*/     H->TheCells[ Pos ].Element = Key;
        /*字符串类型的关键词需要 strcpy 函数!! */
    }
}
```

在开放地址散列表中，**删除操作**要很小心。通常只能“**懒惰删除**”，即需要增加一个“删除标记(**Deleted**)”，而并不是真正删除它。以便查找时不会“**断链**”。其空间可以在下次插入时**重用**。

3. 双散列探测法 (Double Hashing)

❖ d_i 选为 $i \cdot h_2(\text{key})$ ，其中 $h_2(\text{key})$ 是另一个散列函数。我们把它叫做双散列探测法。由此，探测序列成了：
 $h_2(\text{key}), 2h_2(\text{key}), 3h_2(\text{key}), \dots$

 对任意的 key ， $h_2(\text{key}) \neq 0$!

 探测序列还应该保证所有的散列存储单元都应该能够被探测到。选择以下形式有良好的效果：

$$h_2(\text{key}) = p - (\text{key} \bmod p)$$

其中： $p < \text{TableSize}$ ， p 、 TableSize 都是素数。

4. 再散列（Rehashing）

- 开放地址法的装填因子 α ($0 \leq \alpha \leq 1$) 会严重影响查找效率，由于表长在一定时间内是定值， α 与“填入表中的元素个数”成正比。
 - 实用最大装填因子一般取 $0.5 \leq \alpha \leq 0.85$ ，超过最大装填因子将导致查找速度严重下降。
 - 当装填因子过大时，解决的方法是加倍扩大散列表，这样 α 可以减小一半，这个过程叫做“再散列（Rehashing）”。当然，装填因子过小时（比如 $\alpha < 0.3$ ），会浪费空间，此时散列表大小可以减半。
 - 再散列具有偶然性，在交互系统中会使人感觉有“停顿”现象。
 - 在某些系统中，设备的短时“停顿”有可能导致严重的后果。
 - 这些系统应该采用不需要再散列的“分离链接法”。
-

❖ 分离链接法 (Separate Chaining)

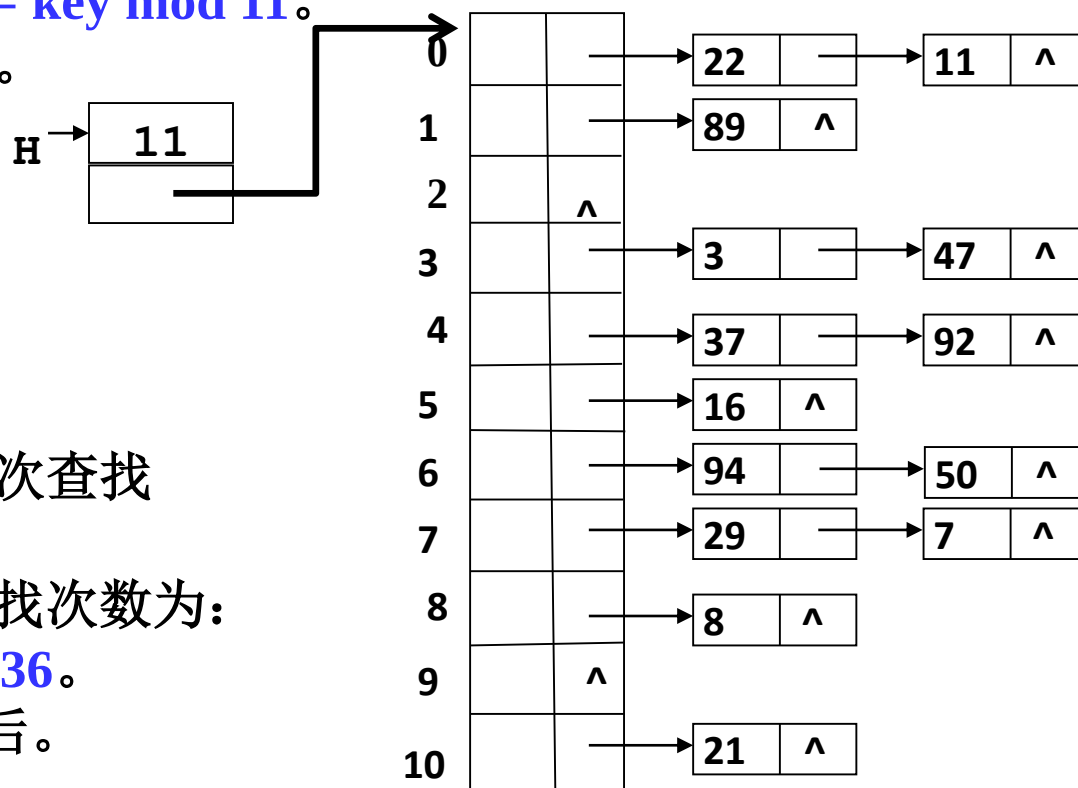
【定义】分离链接法是解决冲突的另一种方法，其做法是将所有关键词为**同义词**的数据对象通过结点链接存储在**同一个单链表中**。

【例5.8】设关键字序列为 47, 7, 29, 11, 16, 92, 22, 8, 3, 50, 37, 89, 94, 21;

➤ 散列函数取为： $h(\text{key}) = \text{key} \bmod 11$ 。

➤ 用分离链接法处理冲突。

```
struct HashTbl
{
    int TableSize;
    List TheLists;
}H;
```



➤ 该表中有9个结点只需1次查找

➤ 5个结点需要2次查找

➤ 因此查找成功的平均查找次数为：

$ASL_s = (9 + 5 \times 2) / 14 \approx 1.36$ 。

ASL_u 估算比较复杂，见后。

```
struct ListNode;  
typedef struct ListNode *Position, *List;  
struct HashTbl;  
typedef struct HashTbl *HashTable;  
struct ListNode  
{  
    ElementType Element;  
    Position Next;  
};
```

```
Position Find( ElementType Key, HashTable H )  
{  
    Position P;  
    List L;  
    /* 1*/    L = &( H->TheLists[ Hash( Key, H->TableSize ) ] );  
    /* 2*/    P = L->Next;  
    /* 3*/    while( P != NULL && strcmp(P->Element, Key) )  
    /* 4*/        P = P->Next;  
    /* 5*/    return P;  
}
```

❖ 散列表的性能分析

- 平均查找长度（**ASL**）用来度量散列表**查找效率**。
 - **另一方面**，关键词的比较次数，取决于产生**冲突的多少**。
 - 影响产生冲突多少有以下**三个因素**：
 - (1) **散列函数是否均匀**；
 - (2) **处理冲突的方法**；
 - (3) 散列表的**装填因子 α** 。
 - 主要分析后两者对查找效率的影响。
-

1. 线性探测法的查找性能

可以证明，线性探测法的期望探测次数满足下列公式：

$$p = \begin{cases} \frac{1}{2} \left[1 + \frac{1}{(1-\alpha)^2} \right] & \text{（对插入和不成功查找而言）} \\ \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) & \text{（对成功查找而言）} \end{cases}$$

➤ 当 $\alpha=0.5$ 时，

插入操作和不成功查找的期望 $ASL_u = 0.5 * (1 + 1/(1-0.5)^2) = 2.5$ 次，
成功查找的期望 $ASL_s = 0.5 * (1 + 1/(1-0.5)) = 1.5$ 次。

➤ 例子5.6的 $\alpha=0.69$ ，于是

期望 $ASL_u = 0.5 * (1 + 1/(1-0.69)^2) = 5.70$ 次

期望 $ASL_s = 0.5 * (1 + 1/(1-0.69)) = 2.11$ 次（例中 $ASL_s = 2.56$ ）。

2. 平方探测法和双散列探测法的查找性能

可以证明，平方探测法和双散列探测法探测次数 满足下列公式：

$$p = \begin{cases} \frac{1}{1-\alpha} & \text{（对插入和不成功查找而言）} \\ -\frac{1}{\alpha} \ln(1-\alpha) & \text{（对成功查找而言）} \end{cases}$$

➤ 当 $\alpha=0.5$ 时，

插入操作和不成功查找的期望 $ASL_u = 1/(1-0.5) = 2$ 次，
成功查找的期望 $ASL_s = -1/0.5 * \ln(1-0.5) \approx 1.39$ 次。

➤ 例子5.7的 $\alpha=0.82$ ，于是

期望 $ASL_u = 1/(1-0.82) \approx 5.56$ 次

期望 $ASL_s = -1/0.5 * \ln(1-0.5) \approx 2.09$ 次（例中 $ASL_s=2$ ）。

❖ 图5.1 表示了期望探测次数与装填因子 α 的关系。

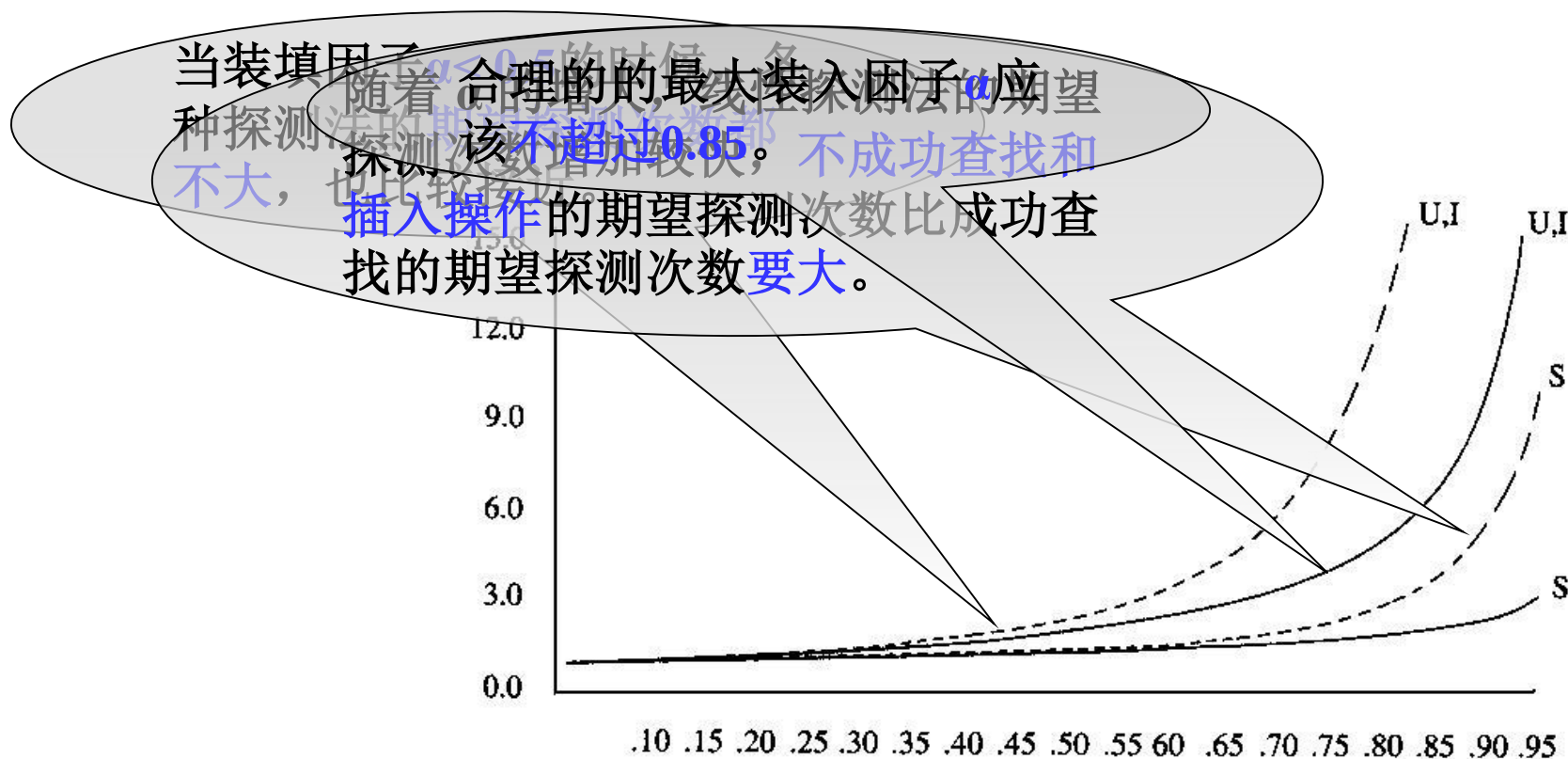


图5.1 线性探测法（虚线）、双散列探测法（实线）
U表示不成功查找，I表示插入，S表示成功查找

3. 分离链接法的查找性能

所有地址链表的平均长度定义成**装填因子** α ， α 有可能超过1。
不难证明：其**期望探测次数** p 为：

$$p = \begin{cases} \alpha + e^{-\alpha} & (\text{对插入和不成功查找而言}) \\ 1 + \frac{\alpha}{2} & (\text{对成功查找而言}) \end{cases}$$

随着 α 增大，**ASLu**增加很慢；
而**ASLs**线性增长。

➤ 当 $\alpha = 1$ 时，
插入操作和不成功查找的**期望 ASLu** $= 1 + e^{-1} = 1.37$ 次，
成功查找的**期望 ASLs** $= 1 + 1/2 = 1.5$ 次。

➤ 例子5.8的14个元素分布在11个单链表中，所以 $\alpha = 14/11 \approx 1.27$ ，故
期望 ASLu $= 1.27 + e^{-1.27} \approx 1.55$ 次
期望 ASLs $= 1 + 1.27/2 \approx 1.64$ 次（例中ASLs = 1.36）

👍 选择合适的 $h(\text{key})$ ，散列法的查找效率期望是常数 $O(1)$ ，它几乎与关键字的空间的大小 n 无关！

👉 它是以较小的 α 为前提。因此，散列方法是一个以空间换时间的成功范例。

👎 散列方法的存储对关键字是随机的，不便于顺序查找关键字，也不适合于范围查找，或最大值最小值查找。

分离链接法：

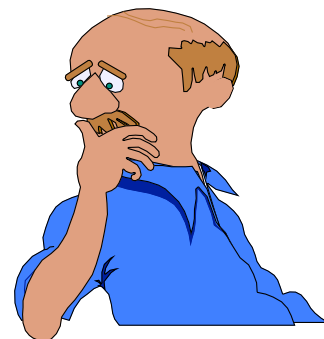
👍 散列表删除不需要“懒惰删除”高法，随机查找有存储“垃圾”。

👎 散列表是顺序存储和链式存储的结合，“链头”部分的存储效率和查找效率都比较高。

👎 关键字的删除只能采用“懒惰删除”法，会导致存储“垃圾”。

👎 太小的 α 可能导致空间浪费，大的 α 又将付出更多的时间代价。不均匀的链表长度导致时间效率的严重下降。

这个问题简单，只要把几本书
扫描一遍就知道答案了！来个难一点的：
你能按照所有单词出现的词频大小
依次输出全部单词吗？



【例5.9】 给定一个英文文本文件，统计文件中所有单词出现的频率，并输出词频最大的前10%的单词及其词频。

为简单起见，假设单词字符定义为大小写字母、数字和下划线，其他字符均认为是单词分隔符，不予考虑。

【分析】 解决这个问题最基本的工作、也是大量的工作是不断对读入的单词在已有单词表中查找，如果已经存在，则将该单词的词频加1，如果不存在，则插入该单词并记词频为1。

核心问题：

如何设计该单词表的数据结构才可以进行快速地查找和插入？



散列表！

❖主控函数

```
int main() {
/* 1 */ int TableSize = 10000; /* 散列表的估计大小 */
    int wordcount = 0, length;
    HashTable H;
    ElementType word;
    FILE *fp;

/* 2 */ char document[30]= "HarryPotter.txt"; /* 要被统计词频的文件名 */
/* 3 */ H = InitializeTable( TableSize ); /* 建立散列表 */
/* 4 */ if(( fp = fopen(document, "r" ))==NULL) FatalError("无法打开文件!\n");
    while( !feof( fp ) ){
/* 5 */     length = GetAWord( fp, word ); /* 从文件中读取一个单词 */
/* 6 */     if(length > 3){ /* 只考虑适当长度的单词 */
/* 7 */         wordcount++;
/* 8 */         InsertAndCount( word, H );
    }
    }
    fclose( fp );
/* 9 */ printf("该文档共出现 %d 个有效单词, ", wordcount);
/* 10 */ Show( H , 10.0/100 ); /* 显示词频前10%的所有单词 */
/* 11 */ DestroyTable( H ); /* 销毁散列表 */
    return 0;
}
```

本片
根据散列表，输出最频繁出现的给定百分比（10%）的单词。